C

Plugin and Skills marketplaces

> **Note**
>
> *How to evaluate any plugin (or skill, or third-party MCP) before installing it, without reading code yourself. You prompt Claude to read the artifact and report. You apply a 7-point rubric to the report.*

The marketplaces (plugins, skills, MCP servers) are the part of the Claude Code ecosystem that grows fastest between print runs and ages worst on the page. By the time you read this, the specific entries available will be different. The shape of *how to evaluate one* changes much more slowly. This appendix is the shape.

I once installed a community plugin without running the rubric below. A week later I noticed an unfamiliar `Stop` hook in my session output that was posting summaries of my work to a server I didn't recognize. The plugin was probably benign: the author later said it was telemetry. I removed it anyway, and I started writing the rubric. You're holding the result.

The book is not anti-community. The book is pro-rubric.

What's in the marketplaces

Three surfaces, three entry points. The same rubric covers all three.

Surface	What it ships	How you browse	What you install into
Plugin marketplace	Bundles of agents, skills, hooks, slash commands, sometimes MCPs	`/plugin` or the published web index	`~/.claude/plugins/`

Surface	What it ships	How you browse	What you install into
Skills (shipped inside plugins)	Single-file skills (the kind Chapter 17 taught you to write)	Skills are distributed inside plugins: use `/plugin` to browse marketplaces that contain skills. Once a plugin is installed, `/skills` lists what's loaded.	`~/.claude/skills/` or `~/work/.claude/skills/`
MCP servers (shipped inside plugins or added directly)	Standalone MCP servers (the kind Chapter 16 taught you to write)	MCP servers ship inside official plugins (see External integrations in the official marketplace via `/plugin`). For one-off MCP installs use `claude mcp add`.	MCPs are added via `claude mcp add` for one-offs, or bundled inside a plugin's `.mcp.json` when shipped via the marketplace.

Plugins are the broadest: they can carry every other surface inside them, which is why the rubric sits in this appendix. Skills and MCPs are narrower; the same rubric still applies, just with fewer points triggering. A standalone skill can't install a hook on your Bash tool; an MCP can't write to your `CLAUDE.md` without permission. The rubric reads the manifest; it cares about the *capabilities granted*, not the surface name.

The trust matrix

Three tiers, three default actions. If you remember nothing else from this appendix, remember the tier you assign when you first see a marketplace entry and the default that comes with it.

Tier	Author	Public history	Default action	Required steps
1	Anthropic-official	n/a	Install	Read the README. Run the rubric on the permissions section only.
2	Known community author with a real GitHub identity, more than six months of public artifacts you've used or seen others use	Yes	Run the rubric, then install if it passes	All seven rubric points. Pin the version.

Tier	Author	Public history	Default action	Required steps
3	Unknown author, fresh account, single-artifact publisher, no public footprint	No	Default no	If you install anyway: rubric + scoped permission profile + a reminder to revoke after one week if you haven't used it

Tier 1 entries get the benefit of the doubt because Anthropic's reputation is the security model. You still read what permissions they ask for, never blindly, but you don't need the full rubric.

Tier 2 is most of the marketplaces by volume. The rubric is mandatory. *Pin the version* matters more than people think: a plugin or skill that auto-updates from a remote can change its behavior after you trusted it.

Tier 3 is the no-by-default category. A new account with one artifact and no other footprint isn't necessarily malicious, but it is necessarily *unknown*. Treat unknown like you'd treat any other code from someone you've never met: scoped, time-boxed, and reversible.

The 7-point rubric

Each point is a question you ask Claude to answer about the artifact. The pattern is the same as the rest of the book: you don't read code; Claude does, and reports back.

1. **What permissions does this ask for?** Read the manifest. Network calls? File-system reads? File-system writes? Shell exec? Writes outside `.claude/`? Anything that asks for `Bash` with a permissive pattern, `Bash(*)` rather than `Bash(specific-command:*)`, is a flag that earns scrutiny.
2. **What MCP servers or external services does it call?** Names and URLs. Anything calling out to a domain you don't recognize is a flag. A weather plugin that calls `weather.com` is fine. A weather plugin that also calls `analytics.[unfamiliar-domain].com` is not.
3. **What hooks does it install, and on what events?** Especially load-bearing: any `PreToolUse` hook on `Bash`, any `SessionStart` hook with network calls, any `Stop` hook that exfiltrates session content. The phone-home plugin I installed lived inside a `Stop` hook. That's the shape to be wary of.
4. **Does it modify your `CLAUDE.md` or `settings.json` outside its own folder?** Marketplace entries should stay in their lane. One that injects instructions into your global `CLAUDE.md`, or rewrites your top-level `settings.json` permissions, is operating beyond its scope. Permitted only with deliberate consent.
5. **What's the update mechanism?** Auto-update from a remote means the code can change after you trust it. Pinned version means the bytes you reviewed are the bytes that run. For Tier 1, auto-update is acceptable. For Tier 2, pin. For Tier 3, never auto-update, and ideally never install at all.

6. **What's the failure mode if the author's account or repo disappears?** If the artifact pulls assets from a remote at runtime, an offline author breaks your `.claude/`. An entry that vendors everything locally fails closed; one with live dependencies fails open in someone else's hands.
7. **Could you reproduce its core function with a 30-line skill of your own?** This is the question that retires more installs than the other six combined. If the answer is yes, the install isn't worth the trust cost. The follow-up prompt is one line: *"could you write me the same thing in 30 lines as a local skill?"* Half the time, Claude can.

The diagnostic prompt

Paste this into a fresh `claude` session, with the marketplace entry's repo URL or local path filled in. The prompt asks Claude to walk the rubric for you.

```
I'm evaluating this before installing it: [PLUGIN / SKILL / MCP URL OR PATH].
Read its manifest, hooks, scripts, and any MCP server config. Produce a report
structured as the 7-point rubric from Appendix C of "Claude Code for
Non-Programmers":
1. Permissions requested (and any wildcard or wide-scope flags)
2. External services or MCPs it calls (with URLs)
3. Hooks installed (event + what they do)
4. Whether it modifies CLAUDE.md or settings.json outside its own folder
5. Update mechanism (auto-update / pinned / unclear)
6. Failure mode if the author's account or repo disappears
7. Whether the core function could be a 30-line local skill instead
Close with: would you install this on your own machine? If no, why not. Be
specific.
```

The output is one page. You read the report, not the code. If anything in the report flags a rubric point, paste that point back to Claude and ask for the line in the manifest that produced the answer. You should be able to verify any concerning answer down to a specific line, without ever reading the surrounding code yourself.

Three worked examples

Tier 1 clean install

An Anthropic-official plugin (any one current at print). The README states permissions plainly: read-only access to a specific tool surface, no network calls, no hooks. You run the rubric on point 1 only. Permissions match the README. You install in one prompt: *"install [PLUGIN-NAME] from the official marketplace and confirm what it added to my .claude/."* Claude installs, reports back with the changed files, you skim the diff, you move on. Total time: under five minutes.

Tier 2 passable install

A **hypothetical** community skill called *fancier-status-line*: a bell-and-whistle that prettifies your terminal status. (*Hypothetical*: this plugin doesn't exist; it stands in for the shape of a Tier 2 install.) The author has six months of

GitHub history, three other artifacts you've heard of, and a real identity. You run the diagnostic prompt. The report shows: read-only filesystem permissions, no external network calls, one `SessionStart` hook that updates a status string, no modifications outside its own folder, version pinned in the manifest. Rubric clears. You install, pinning the version Claude reviewed. You note the pin in your `settings.json` annotation file (*Appendix F*). Total time: under fifteen minutes.

Tier 3 failed evaluation

A **hypothetical** plugin called *inbox-supercharger*: promises to triple your Iris throughput. (*Hypothetical*: this plugin doesn't exist; it stands in for the shape of a Tier 3 fail.) Author: a four-week-old GitHub account with one artifact. You run the diagnostic prompt. The report surfaces a `Stop` hook that POSTs session summaries to a third-party domain "for analytics." The rubric fails on point 2 (unknown external service), point 3 (Stop hook exfiltrates session content), and point 4 (modifies your global `CLAUDE.md` to add a "share telemetry" line).

You don't install. You note the author for future reference. The thirty seconds you spent on the diagnostic prompt is the punchline of this whole appendix.

Skills-specific notes

A skill is narrower than a plugin, usually a single markdown file with frontmatter and instructions. The rubric still applies, but two points carry most of the weight for skills:

- **Point 1 (permissions)**: a skill that requests `Bash` access at all is rare. If it does, ask why. Most useful skills are pure-prompt instructions that need no permissions beyond what your session already has.
- **Point 7 (could a 30-line skill replace this)**: for skills, this point folds in on itself. If the answer is *"yes, but it's already a 30-line skill,"* then you're fine; copy it into `~/work/.claude/skills/` directly and skip the marketplace install layer entirely. A skill is small enough to read top-to-bottom; you don't need a manifest abstraction the way you do for a plugin.

The rule of thumb: read the skill file. If it's under 100 lines and you can name what it does after one read, install. If it sprawls across multiple files with shell scripts, treat it as a plugin and run the full rubric.

Out of scope (cross-links)

Authoring your own: *Chapter 16* (MCP) and *Chapter 17* (skill); the companion repo's engineer guide points at the official authoring spec. **Cryptographic signature verification** is a moving target; the marketplace's current signing guidance is authoritative, and this rubric is complementary to it (signatures don't catch an authentic plugin whose author added telemetry in v1.4). **Scoped permission profiles** for Tier 3 installs are the named profiles from *Chapter 10*: *Appendix F* shows the shape. **When a marketplace entry breaks something**, *Appendix E* indexes by symptom; marketplace entries are surprisingly often the cause of *"permission denied unexpectedly."*

The marketplaces will keep growing. The rubric will not.